Εφαρμογή Διαχείρισης Εκδηλώσεων και Ηλεκτρονικών Κρατήσεων

Μάθημα: Τεχνολογίες Εφαρμογών Διαδικτύου
Τμήμα: Πληροφορικής & Τηλεπικοινωνιών, Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών

Ομάδα Υλοποίησης:

1. ΝΙΚΟΛΑΟΣ ΚΟΥΤΣΟΥΝΗΣ (Α.Μ.: 1115202300091)
2. ΒΟΪΣΤΡΙΚΟΒ ΝΙΚΗΤΑ (Α.Μ.: 1115202300019)

---

1. Εισαγωγή

Η εφαρμογή ακολουθεί την αρχιτεκτονική πελάτη-εξυπηρετητή (Client-Server / Web Browser - Web Server) και υποστηρίζει τέσσερις (4) διακριτούς ρόλους χρηστών:

- **Επισκέπτης (Visitor / Guest):** Έχει τη δυνατότητα να περιηγηθεί στη σελίδα καλωσορίσματος, να αναζητήσει εκδηλώσεις με πολλαπλά κριτήρια και να δει τις λεπτομέρειές τους, χωρίς όμως να μπορεί να κάνει κράτηση.
- **Συμμετέχων (Attendee):** Πιστοποιημένος χρήστης που μπορεί να πραγματοποιήσει κρατήσεις για εκδηλώσεις, να δει το ιστορικό των κρατήσεών του, να επικοινωνήσει με διοργανωτές μέσω μηνυμάτων και να λαμβάνει εξατομικευμένες προτάσεις εκδηλώσεων.
- **Διοργανωτής (Organizer):** Μπορεί να δημιουργεί νέες εκδηλώσεις (ορίζοντας κατηγορίες, τοποθεσία στον χάρτη, εισιτήρια και χωρητικότητα), να τις τροποποιεί, να παρακολουθεί τις κρατήσεις σε πραγματικό χρόνο, να επικοινωνεί με τους συμμετέχοντες και να ακυρώνει εκδηλώσεις ειδοποιώντας αυτόματα όλους τους κρατήσαντες.
- **Διαχειριστής Εφαρμογής (Admin):** Προεγκατεστημένος λογαριασμός συστήματος με δικαίωμα εξέτασης, έγκρισης ή απόρριψης νέων εγγραφών χρηστών, καθώς και εξαγωγής όλων των εκδηλώσεων σε μορφή XML (βάσει του προτύπου DTD της εκφώνησης) και JSON.

---

2. Σχεδιαστικές Αποφάσεις και Αρχιτεκτονική

2.1 Τεχνολογικό Stack

- **Backend:** Node.js (v20+) με Express framework σε γλώσσα **TypeScript**.
  - **Prisma ORM:** Χρησιμοποιείται για την απεικόνιση του σχεσιακού μοντέλου της βάσης σε αντικειμενοστρεφές, παρέχοντας ασφαλή ερωτήματα τύπων (type-safe queries) και αυτόματες μεταναστεύσεις (migrations).
  - **Authentication & Security:** JSON Web Tokens (access tokens & refresh tokens σε HTTP-only cookies), `bcrypt` για το hashing των κωδικών πρόσβασης (salt rounds = 12), `helmet` για HTTP

security headers και `cors` για έλεγχο προέλευσης αιτημάτων.
    - **Data Validation:** Βιβλιοθήκη `zod` για αυστηρό έλεγχο εγκυρότητας όλων των εισερχόμενων δεδομένων (payloads) στα endpoints.
- **Frontend:** Single Page Application (SPA) βασισμένη σε **React** και **Vite** με TypeScript.
    - **UI & Styling:** Tailwind CSS για σύγχρονη και responsive εμφάνιση σε όλες τις συσκευές.
    - **State Management:** `zustand` για απλή, αποδοτική διαχείριση της κατάστασης σύνδεσης του χρήστη (auth store).
    - **Maps:** Βιβλιοθήκη `leaflet` και `react-leaflet` με tiles από το OpenStreetMap για την προβολή και επιλογή γεωγραφικών συντεταγμένων.
    - **Routing:** `react-router-dom` με προστατευμένα routes ανά ρόλο χρήστη (`RequireAuth`, `RequireRole`).
- **Βάση Δεδομένων: PostgreSQL 16** (Alpine Linux container) με πλήρη υποστήριξη συναλλαγών (ACID transactions) για ασφαλείς κρατήσεις θέσεων.
- **Εργαλεία Διαχείρισης: pgAdmin 4** ενσωματωμένο σε container για εύκολη επισκόπηση και έλεγχο των πινάκων της βάσης.

2.2 Υλοποίηση HTTPS και Ασφάλεια (SSL/TLS)

1. **Δημιουργία Πιστοποιητικών (`scripts/generate-certs.sh`):**
   Δημιουργήσαμε ένα αυτοματοποιημένο bash script που χρησιμοποιεί το OpenSSL για να παράγει ιδιωτικό κλειδί RSA 2048-bit (`server.key`) και αυτο-υπογεγραμμένο πιστοποιητικό X.509 (`server.crt`) διάρκειας 365 ημερών:

   ```
   openssl req -x509 -newkey rsa:2048 -nodes \
     -keyout certs/server.key \
     -out    certs/server.crt \
     -days 365 \
     -subj "/C=GR/ST=Attica/L=Athens/O=TEDI/CN=localhost" \
     -addext "subjectAltName=DNS:localhost,DNS:backend,IP:127.0.0.1"
   ```

2. **Backend HTTPS Server (`backend/src/index.ts`):**
   Ο εξυπηρετητής Express δεν τρέχει σε απλό HTTP. Διαβάζει τα πιστοποιητικά από τις διαδρομές `SSL_KEY_PATH` και `SSL_CERT_PATH` και εκκινεί έναν native Node.js HTTPS server μέσω του `https.createServer`:

   ```
   const httpsOptions = {
     key:  fs.readFileSync(env.SSL_KEY_PATH),
     cert: fs.readFileSync(env.SSL_CERT_PATH),
   };
   https.createServer(httpsOptions, app).listen(env.PORT);
   ```

3. **Frontend HTTPS Server (`frontend/vite.config.ts`):**
   Ο αναπτυξιακός server του Vite ρυθμίστηκε ώστε να σερβίρει την εφαρμογή React απευθείας μέσω HTTPS στη θύρα 5173, φορτώνοντας το ίδιο ζεύγος κλειδιού/πιστοποιητικού (`server.key` και `server.crt`).

3. Σχήμα Βάσης Δεδομένων & Μοντέλο

- **User (`users`):** Αποθηκεύει τα στοιχεία των χρηστών: μοναδικό `id` (cuid), `username` (unique), `passwordHash`, `firstName`, `lastName`, `email` (unique), `phone`, `address`, `city`, `country`, συντεταγμένες `latitude` & `longitude`, `taxId` (ΑΦΜ), `role` (`ATTENDEE`, `ORGANIZER`, `ADMIN`) και `status` (`PENDING`, `APPROVED`, `REJECTED`).
- **RefreshToken (`refresh_tokens`):** Αποθηκεύει τα hashes των refresh tokens για ασφαλή ανανέωση των JWT access tokens χωρίς διαρκή εισαγωγή κωδικού.
- **Category (`categories`):** Περιέχει τις διαθέσιμες κατηγορίες εκδηλώσεων (π.χ. Μουσική, Θέατρο, Συνέδριο).
- **Event (`events`):** Το κεντρικό μοντέλο της εκδήλωσης: `title`, `eventType`, `venue`, `address`, `city`, `country`, `latitude`, `longitude`, `startDateTime`, `endDateTime`, συνολική χωρητικότητα `capacity`, κατάσταση `status` (`DRAFT`, `PUBLISHED`, `COMPLETED`, `CANCELLED`), περιγραφή `description` και αναφορά στον διοργανωτή `organizerId`.
- **EventCategory (`event_categories`):** Πίνακας σύνδεσης πολλά-προς-πολλά (N:M) μεταξύ `Event` και `Category`, καθώς κάθε εκδήλωση μπορεί να ανήκει σε πολλαπλές κατηγορίες σύμφωνα με το DTD (`Category+`).
- **TicketType (`ticket_types`):** Οι διαφορετικοί τύποι εισιτηρίων ανά εκδήλωση (π.χ. Γενική Είσοδος, Φοιτητικό, VIP). Αποθηκεύει `name`, `price`, συνολικό πλήθος `quantity` και διαθέσιμο υπόλοιπο `available`.
- **Booking (`bookings`):** Καταγράφει τις κρατήσεις: συνδέει τον χρήστη (`userId`), την εκδήλωση (`eventId`) και τον επιλεγμένο τύπο εισιτηρίου (`ticketTypeId`), αποθηκεύοντας το πλήθος εισιτηρίων `numberOfTickets`, το συνολικό κόστος `totalCost` και την κατάσταση `status` (`CONFIRMED`, `CANCELLED`, `PENDING`).
- **EventPhoto (`event_photos`):** Αποθηκεύει τα ονόματα αρχείων εικόνων για κάθε εκδήλωση (`filename`, `isMain`, `sortOrder`), καλύπτοντας το στοιχείο `Media` του DTD.
- **EventView (`event_views`):** Καταγράφει τις επισκέψεις των χρηστών στις σελίδες λεπτομερειών εκδηλώσεων (`userId`, `eventId`, `viewedAt`). Τα δεδομένα αυτά τροφοδοτούν το σύστημα συστάσεων για την κατανόηση των ενδιαφερόντων του χρήστη ακόμη και πριν κάνει κράτηση.
- **Message (`messages`):** Υποστηρίζει την εσωτερική επικοινωνία: `senderId`, `recipientId`, `subject`, `body`, κατάσταση ανάγνωσης `read`, και πεδία ανεξάρτητης διαγραφής `deletedBySender` και `deletedByRecipient`.

4. Υλοποίηση

4.1 Ρόλοι Χρηστών & Έγκριση Εγγραφής

1. **Εγγραφή (Sign Up):** Ο χρήστης συμπληρώνει όλα τα απαραίτητα στοιχεία (username, password, ονοματεπώνυμο, email, τηλέφωνο, διεύθυνση, ΑΦΜ, γεωγραφικές συντεταγμένες). Το σύστημα

ελέγχει τη μοναδικότητα του username και του email.

2. **Κατάσταση Εκκρεμότητας (Pending Approval):** Κάθε νέος χρήστης αρχικοποιείται με `status = PENDING`. Μετά την εγγραφή ανακατευθύνεται στη σελίδα αναμονής έγκρισης και δεν μπορεί να συνδεθεί στις υπόλοιπες λειτουργίες μέχρι να εγκριθεί.
3. **Ενσωματωμένος Διαχειριστής:** Κατά την εγκατάσταση (seed), δημιουργείται αυτόματα ο λογαριασμός διαχειριστή (`username: admin`, `password: 123`).
4. **Διαχείριση Χρηστών (Admin Panel):** Ο διαχειριστής βλέπει τη λίστα όλων των χρηστών, μπορεί να ανοίξει την καρτέλα αναλυτικών στοιχείων κάθε χρήστη (συμπεριλαμβανομένου του ΑΦΜ και των στοιχείων επικοινωνίας) και να πατήσει **Έγκριση (Approve)** ή **Απόρριψη (Reject)**.

4.2 Διαχείριση Εκδηλώσεων, Χωρητικότητας & Εισιτηρίων

- **Δημιουργία Εκδήλωσης:** Ο διοργανωτής εισάγει τα στοιχεία της εκδήλωσης, επιλέγει πολλαπλές κατηγορίες, ορίζει τοποθεσία πάνω στον διαδραστικό χάρτη, ανεβάζει φωτογραφίες και καθορίζει τους τύπους εισιτηρίων (όνομα, τιμή, ποσότητα).
- **Έλεγχος Χωρητικότητας (Capacity Invariant):**
  Τόσο στο frontend όσο και με αυστηρή επαλήθευση στο backend, ελέγχεται ότι: $$\sum \text{quantity}_{\text{ticket_type}} \le \text{Capacity}_{\text{event}}$$ Δεν επιτρέπεται η δημιουργία ή ενημέρωση εισιτηρίων αν το άθροισμά τους ξεπερνά τη δηλωμένη χωρητικότητα της εκδήλωσης.
- **Έλεγχος Διαθεσιμότητας στις Κρατήσεις:**
  Κατά την κράτηση, ελέγχεται σε επίπεδο συναλλαγής βάσης δεδομένων ότι τα ζητούμενα εισιτήρια είναι διαθέσιμα (`available >= requested`). Με την ολοκλήρωση της κράτησης, μειώνεται αυτόματα το `available` του τύπου εισιτηρίου και δεσμεύονται οι θέσεις.

4.3 Πολιτική Ακύρωσης & Διαγραφής Εκδηλώσεων

Σύμφωνα με τους κανόνες της εκφώνησης:

- **Διαγραφή (Delete):** Επιτρέπεται **μόνο** όταν η εκδήλωση βρίσκεται σε κατάσταση προσχεδίου (`DRAFT`) ή πριν πραγματοποιηθεί η πρώτη κράτηση. Αν διαγραφεί, αφαιρούνται όλα τα σχετικά δεδομένα και αρχεία εικόνων.
- **Ακύρωση (Cancel):** Όταν μια εκδήλωση έχει δημοσιευτεί (`PUBLISHED`) και έχουν ήδη υποβληθεί κρατήσεις, η διαγραφή απαγορεύεται. Ο διοργανωτής μπορεί μόνο να αλλάξει την κατάσταση σε `CANCELLED`.
  - Όλα τα αποθηκευμένα δεδομένα και το ιστορικό κρατήσεων διατηρούνται άθικτα στη βάση για λόγους ιχνηλασιμότητας.
  - Απαγορεύεται οποιαδήποτε νέα κράτηση σε ακυρωμένη εκδήλωση.

4.4 Αναζήτηση, Φίλτρα & Χάρτες OpenStreetMap

- **Αναζήτηση & Πλοήγηση:** Οι χρήστες και οι επισκέπτες μπορούν να αναζητήσουν εκδηλώσεις με βάση:
  - Ελεύθερο κείμενο (αναζήτηση σε τίτλο και περιγραφή)
  - Κατηγορία εκδήλωσης

    - Χρονικό διάστημα (από/έως ημερομηνία)
    - Εύρος τιμής εισιτηρίου (ελάχιστη / μέγιστη τιμή)
    - Τοποθεσία / Πόλη
    - Τα αποτελέσματα παρουσιάζονται με σελιδοποίηση (pagination) για άμεση και γρήγορη φόρτωση.
- **Χάρτης OpenStreetMap (Leaflet):** Στη σελίδα αναλυτικών στοιχείων κάθε εκδήλωσης προβάλλεται διαδραστικός χάρτης OpenStreetMap με marker στις ακριβείς συντεταγμένες της εκδήλωσης.
- **Επιβεβαίωση Κράτησης:** Πριν από την οριστικοποίηση της κράτησης, εμφανίζεται παράθυρο επιβεβαίωσης στον χρήστη, καθώς η κράτηση είναι οριστική και δεν μπορεί να αναιρεθεί αυτόνομα.

4.5 Σύστημα Μηνυμάτων & Μαζική Ειδοποίηση Ακύρωσης

- **Επικοινωνία Συμμετέχοντα - Διοργανωτή:** Μετά την πραγματοποίηση κράτησης, ανοίγει κανάλι επικοινωνίας μεταξύ των δύο πλευρών.
- **Οργάνωση Μηνυμάτων:** Τα μηνύματα διαχωρίζονται σε καρτέλες **Εισερχόμενα (Inbox)** και **Απεσταλμένα (Sent)**.
- **Ένδειξη Νέων Μηνυμάτων (Unread Badge):** Στη μπάρα πλοήγησης (Navbar) εμφανίζεται ένδειξη (badge) με τον αριθμό των αδιάβαστων μηνυμάτων σε πραγματικό χρόνο.
- **Διαγραφή Μηνυμάτων:** Υποστηρίζεται ανεξάρτητη διαγραφή (soft-delete ανά χρήστη), ώστε η διαγραφή από τα απεσταλμένα του ενός να μην εξαφανίζει το μήνυμα από τα εισερχόμενα του άλλου.
- **Μαζική Ειδοποίηση Ακύρωσης (Broadcast Message):** Σε περίπτωση που μια εκδήλωση ακυρωθεί, ο διοργανωτής έχει τη δυνατότητα να στείλει με ένα κλικ ενημερωτικό μήνυμα προς όλους τους χρήστες που έχουν ενεργή κράτηση για τη συγκεκριμένη εκδήλωση.

4.6 Εξαγωγή Δεδομένων σε XML (βάσει DTD) & JSON

Ο διαχειριστής μπορεί από το Admin Panel να εξάγει το σύνολο των εκδηλώσεων:

- **Εξαγωγή XML:** Το αρχείο παράγεται δυναμικά από το backend (`/admin/export/events/xml`) και ακολουθεί **κατά γράμμα** το Document Type Definition (DTD) που ορίζει η εκφώνηση της εργασίας:
    - Στοιχείο ρίζας `<Events>` που περιέχει στοιχεία `<Event EventID="...">` .
    - Όλα τα υποστοιχεία: `<Title>` , `<Category>` (πολλαπλά), `<EventType>` , `<Venue>` , `<Address>` , `<City>` , `<Country>` , `<GeoLocation Latitude="..." Longitude="..."/>` , `<StartDateTime>` , `<EndDateTime>` , `<Capacity>` .
    - Δομημένα `<TicketTypes>` με υποστοιχεία `<TicketType TicketTypeID="...">` (`<Name>` , `<Price>` , `<Quantity>` , `<Available>`).
    - Δομημένα `<Bookings>` με υποστοιχεία `<Booking BookingID="...">` (`<Attendee UserID="...">` , `<Time>` , `<TicketTypeRef>` , `<NumberOfTickets>` , `<TotalCost>` , `<BookingStatus>`).
    - `<Organizer UserID="...">` , `<Status>` , `<Description>` , και `<Media>` με στοιχεία `<Photo>` .

- **Εξαγωγή JSON:** Αντίστοιχο endpoint (`/admin/export/events/json`) που εξάγει την πλήρη ιεραρχική δομή των εκδηλώσεων σε μορφή JSON.

---

5. Αλγόριθμος Συστάσεων (Biased Matrix Factorization)

5.1 Πώς Λειτουργεί ο Αλγόριθμος

Ο αλγόριθμος **Biased Matrix Factorization** προσπαθεί να μαντέψει πόσο θα ενδιέφερε μια εκδήλωση έναν συγκεκριμένο χρήστη. Για να υπολογίσει αυτή την πρόβλεψη, συνδυάζει τρία βασικά στοιχεία:

1. **Τον γενικό μέσο όρο:** Ποιο είναι το συνηθισμένο επίπεδο ενδιαφέροντος στην εφαρμογή συνολικά.
2. **Τις γενικές συνήθειες (Biases):**
   - **Του χρήστη:** Κάποιοι χρήστες είναι πιο ενεργοί και κάνουν εύκολα κρατήσεις, ενώ άλλοι είναι πιο επιφυλακτικοί.
   - **Της εκδήλωσης:** Κάποιες εκδηλώσεις (π.χ. μια μεγάλη συναυλία) είναι από τη φύση τους δημοφιλείς σε όλους, ενώ άλλες απευθύνονται σε μικρότερο κοινό.
3. **Το ταίριασμα στα γούστα (Factors):**
   - Το σύστημα αναθέτει σε κάθε χρήστη και σε κάθε εκδήλωση 10 «κρυφά χαρακτηριστικά» (π.χ. είδος μουσικής, στιλ εκδήλωσης, τοποθεσία).
   - Όσο περισσότερο ταιριάζουν τα προσωπικά γούστα του χρήστη με το προφίλ της εκδήλωσης, τόσο μεγαλύτερη βαθμολογία προκύπτει.

5.2 Πώς Εκπαιδεύεται το Μοντέλο (SGD)

Για να μάθει το σύστημα τις προτιμήσεις των χρηστών, χρησιμοποιεί τη μέθοδο **Στοχαστικής Καθόδου Κλίσης (SGD - Stochastic Gradient Descent)**. Η λογική της λειτουργεί βήμα-βήμα:

1. **Αρχικό ξεκίνημα:** Στην αρχή το σύστημα δεν γνωρίζει τίποτα. Ξεκινά δίνοντας μικρές, τυχαίες τιμές στα χαρακτηριστικά κάθε χρήστη και κάθε εκδήλωσης.
2. **Υπολογισμός του λάθους:** Παίρνει μία-μία τις πραγματικές ενέργειες (κρατήσεις ή επισκέψεις), κάνει μια δοκιμαστική πρόβλεψη και μετράει πόσο «έξω έπεσε» σε σχέση με την πραγματικότητα.
3. **Μικρές διορθώσεις (Learning Rate):**
   - Αν η πρόβλεψη ήταν λάθος, διορθώνει ελαφρώς τα χαρακτηριστικά του χρήστη και της εκδήλωσης προς τη σωστή κατεύθυνση.
   - Η αλλαγή γίνεται με πολύ μικρά και προσεκτικά βήματα (**ρυθμός μάθησης / learning rate**), ώστε να συγκλίνει σταδιακά χωρίς απότομα άλματα.
4. **Αποφυγή υπερβολών (Regularization):** Ταυτόχρονα εφαρμόζεται ένας συντελεστής «φρεναρίσματος» (**regularization**), ο οποίος εμποδίζει τα νούμερα από το να γίνουν υπερβολικά μεγάλα, διατηρώντας το μοντέλο ισορροπημένο.

5.3 Αποφυγή Υπερπροσαρμογής & Early Stopping

Επειδή οι πίνακες αλληλεπιδράσεων είναι εξαιρετικά αραιοί, καθώς κάθε χρήστης συνήθως αλληλεπιδρά με ελάχιστες εκδηλώσεις, αν ο αλγόριθμος γίνει για 100 εποχές, θα οδηγησει σε overfitting.

Για την επίλυση αυτού του προβλήματος υλοποιήσαμε μηχανισμό **Early Stopping με Validation Split**:

1. **Διαχωρισμός Επικύρωσης (Validation Split):** Για κάθε χρήστη με τουλάχιστον 3 αλληλεπιδράσεις, απομονώνουμε την τελευταία αλληλεπίδραση σε ένα validation set (μορφή leave-one-out). Χρήστες με λιγότερες από 3 αλληλεπιδράσεις παραμένουν εξ ολοκλήρου στο training set ώστε να μην απορρίπτονται αραιοί χρήστες.
2. **Παρακολούθηση Σφάλματος (Validation RMSE):** Στο τέλος κάθε εποχής εκπαίδευσης υπολογίζεται το ριζικό μέσο τετραγωνικό σφάλμα (Root Mean Squared Error) στο validation set.
3. **Υπομονή Τερματισμού (`earlyStoppingPatience = 5`):** Αν το σφάλμα επικύρωσης βελτιωθεί, κρατάμε στιγμιότυπο των βαρών (`snapshot`). Αν για 5 συνεχόμενες εποχές δεν σημειώσει καμία βελτίωση, η εκπαίδευση διακόπτεται πρόωρα και επαναφέρονται αυτόματα τα καλύτερα βάρη που παρατηρήθηκαν (`restore`).
4. **Προστασία Μικρού Μεγέθους (`minValidationSize = 20`):** Αν το validation set περιέχει λιγότερα από 20 σημεία (π.χ. σε μια φρέσκια βάση δεδομένων με λίγους χρήστες), το σήμα επικύρωσης είναι υπερβολικά θορυβώδες για να βασιστεί απόφαση διακοπής. Σε αυτή την περίπτωση, το σύστημα εκπαιδεύεται με ασφάλεια για προκαθορισμένο αριθμό εποχών χωρίς early stopping.

5.4 Πειραματική Αξιολόγηση στο Dataset του e-class

Για την αξιολόγηση του αλγορίθμου, χρησιμοποιήθηκε το επίσημο dataset του μαθήματος από το e-class (`event_interest.csv`), το οποίο περιέχει πραγματικές ιστορικές αλληλεπιδράσεις χρηστών με εκδηλώσεις.

Η αξιολόγηση υλοποιήθηκε στο script `evaluate-recommender.ts`. Η αντιστοίχιση των σημάτων του dataset έγινε ως εξής:

- `interested = 1` → βαθμολογία 5.0
- `not_interested = 1` → βαθμολογία 1.0
- λοιπές προσκλήσεις / προβολές → βαθμολογία 2.0

Αποτελέσματα Αξιολόγησης:

```
Dataset: backend/dataset/rel_event_csvs/event_interest.csv
Interactions: 13,983 (1,970 users, 8,119 events)
Skipped rows without user/event identifiers: 843
Signals: 3,810 interested, 463 not interested, 9,710 viewed/neutral
Split: 12,017 training, 1,966 test (Leave-One-Out split)
BMF RMSE: 1.3508
BMF MAE:  1.1819
Global-mean baseline RMSE: 1.3909
```

Τα αποτελέσματα αποδεικνύουν ότι ο αλγόριθμος Biased Matrix Factorization πετυχαίνει στατιστικά σημαντική μείωση του σφάλματος πρόβλεψης σε σύγκριση με το baseline του παγκόσμιου μέσου όρου (RMSE 1.3508 έναντι 1.3909).

---

6. Οδηγίες Εγκατάστασης και Εκτέλεσης (How to Run)

A. `.env`

```
JWT_SECRET=654df16b2087abb16da7e968ba7fb6cdf2a54b94289537f92038f4111f0e93d2

DB_USER=events_user
DB_PASSWORD=123
DB_NAME=events_db

PGADMIN_EMAIL=admin@example.com
PGADMIN_PASSWORD=123

# User / Group IDs for Linux file permissions
LOCAL_UID=1000
LOCAL_GID=1000
```

B. TLS/SSL Certificates: `certs/server.key` & `certs/server.crt`

Run:

```
chmod +x scripts/generate-certs.sh
./scripts/generate-certs.sh
```

This generates `certs/server.key` and `certs/server.crt` valid for 365 days with Subject Alternative Names for `localhost`, `backend`, and `127.0.0.1`.

---

2. Method 1: Docker Compose (Recommended)

This starts PostgreSQL, Backend, Frontend, and pgAdmin in containers with all networking and volumes configured.

Step 1: Start all containers

```
docker compose up -d --build
```

• The backend container automatically executes `npx prisma generate` and `npx prisma migrate deploy` on startup.

Step 2: Seed the Database

Run the Prisma seed script inside the running backend container:

```
docker compose exec backend npm run db:seed
```

This will populate the database with default accounts (admin, organizers, attendees), event categories, test events, ticket tiers, images, and recommendation data.

Step 3: Access the Application

Open your browser to:

• **Frontend Application:** https://localhost:5173

> **Browser Warning:** Because certificates are self-signed, your browser will show an "Untrusted certificate" or "Your connection is not private" warning. Click **Advanced → Proceed to localhost (unsafe)**.
> *(Note: All API routes `/auth`, `/events`, etc. are proxied through port 5173, so you only have to accept this certificate once.)*

---

3. Method 2: Τοπική Εκτέλεση χωρίς Docker (Local / Bare Metal)

Αν επιθυμείτε να τρέξετε την εφαρμογή τοπικά χωρίς Docker, απαιτείται εγκατεστημένο **Node.js (v20+)** και **PostgreSQL**:

1. **Εκκίνηση της PostgreSQL** και δημιουργία της βάσης `events_db` με τα στοιχεία του `.env`.
2. **Backend Setup:**

   ```
   cd backend
   npm install
   npx prisma generate
   npx prisma migrate deploy
   npm run db:seed
   npm run dev
   ```

   Το backend ξεκινάει σε HTTPS στο `https://localhost:3000`.
3. **Frontend Setup (σε δεύτερο τερματικό):**

   ```
   cd frontend
   npm install
   npm run dev
   ```

   Το frontend ξεκινάει σε HTTPS στο `https://localhost:5173`.

---

4. Προκαθορισμένοι Λογαριασμοί Δοκιμών (Demo Accounts)

Με την εκτέλεση του `db:seed`, δημιουργούνται οι παρακάτω λογαριασμοί με κοινό κωδικό πρόσβασης **`123`**:

Όνομα Χρήστη (Username)	Ρόλος (Role)	Περιγραφή
`admin`	`ADMIN`	Διαχειριστής συστήματος (πρόσβαση σε έγκριση χρηστών & XML/JSON export)
`organizer`	`ORGANIZER`	Διοργανωτής (δημιουργία, διαχείριση εκδηλώσεων & μηνύματα)
`customer`	`ATTENDEE`	Συμμετέχων (αναζήτηση, κρατήσεις εισιτηρίων & συστάσεις)
`customer2` - **`customer5`**	`ATTENDEE`	Επιπλέον συμμετέχοντες με ιστορικό κρατήσεων για δοκιμή των συστάσεων

---

5. Αξιολόγηση Αλγορίθμου Συστάσεων (Dataset Evaluation Script)

Για να τρέξετε το πείραμα αξιολόγησης του Biased Matrix Factorization πάνω στο επίσημο dataset του e-class:

- **Μέσω Docker:**

  ```
  docker compose exec backend npm run dataset:evaluate
  ```

- **Τοπικά (μέσα από τον φάκελο `backend`):**

  ```
  npm run dataset:evaluate
  ```